

Firestore en Inventario iOS

Como se implemento la bitacora de auditoria con Realtime Database. Recorrido por el codigo, archivo por archivo.

Ferreteria Zamora — App de gestion de inventario

Proyecto: inventario-ios | SDK: firebase-ios-sdk 12.18.0 | Producto: FirebaseDatabase

Que vas a encontrar aca

Este documento explica **por que** la app usa Firestore, **que** guarda ahi y **como** esta escrito el codigo que lo hace. Cada seccion muestra el codigo real del proyecto y lo desarma linea por linea.

El orden sigue el camino que recorre un dato: primero se instala el SDK, despues se configura, se modela el dato, se escribe, se lee, y por ultimo se muestra en pantalla.

Antes de empezar. La app ya tenia un backend propio (Spring Boot) y una base local (Core Data). Firestore no reemplaza a ninguno de los dos: se suma para guardar una cosa que los otros dos no guardan. Entender esa division es la clave de todo lo demas.

1. El problema: que guardar en Firestore

La app ya tenia dos lugares donde viven los datos:

- **El backend de Spring** (una API REST): es la fuente de verdad de productos, categorias, proveedores y usuarios.
- **Core Data** (base local en el telefono): copia local de todo eso, para que la app funcione sin conexion. Un SyncManager las mantiene sincronizadas.

Meter en Firestore una copia de los productos habria sido un error: quedarian **dos fuentes de verdad para el mismo dato**, y cuando difieran no hay forma de saber cual tiene razon.

Entonces la pregunta correcta no es *"que copio a Firestore"* sino *"que informacion no tiene hoy ningun lado"*. Y hay una respuesta clara: **el historial**. El backend guarda que un producto *hoy* cuesta \$/ 45, pero no guarda quien le puso ese precio, cuando, ni cual era antes.

La entidad elegida: bitacora de auditoria. Cada alta, edicion y baja de producto, categoria o proveedor deja una entrada con que se hizo, sobre que, quien y cuando. Es informacion nueva, no duplicada, y ademas es *append-only*: solo se agrega, nunca se edita ni se borra. Eso hace la implementacion mucho mas simple, porque no hay updates, ni conflictos, ni cola offline que resolver.

Por que Realtime Database y no Firestore

Firestore ofrece dos bases distintas. Se eligio **Realtime Database** porque el dato es una lista plana de eventos que solo crece. Firestore aporta consultas complejas y subcolecciones que aca no hacen falta, y su modelo de documentos es mas verboso para algo tan simple. Son productos separados dentro de la misma consola: si abris Firestore, la base va a estar vacia.

2. Mapa de la implementacion

Toda la integracion son **cinco archivos nuevos** y unos pocos retoques. Esta es la lista completa:

Archivo	Que hace
App/AppDelegate.swift	Arranca Firebase al abrir la app (2 lineas)
Resources/GoogleService-Info.plist	Configuracion que genera la consola de Firebase
Models/Bitacora/EventoBitacora.swift	El dato: que campos tiene un evento
Services/BitacoraService.swift	Escribe y lee contra Firebase
Features/Bitacora/BitacoraListViewModel.swift	Pide los datos y avisa a la pantalla
Features/Bitacora/BitacoraListViewController.swift	Dibuja la lista
(6 ViewModels ya existentes)	Llaman a registrar() cuando el usuario modifica algo

El flujo completo, de punta a punta:

```
El usuario guarda una categoria
|
v
CategoriaFormViewModel.guardar()      <- logica que ya existia
|
+--> service.crear(...)               --> Core Data (y luego Spring)
|
+--> BitacoraService.registrar()      --> Firebase
|
v
Firebase Realtime Database
|
v
BitacoraListViewModel.cargar() <-- BitacoraService.todos()
|
v
BitacoraListViewController (la pantalla)
```

Fijate en la bifurcacion. Guardar en Core Data y registrar en Firebase son dos caminos independientes. Si Firebase falla, la categoria igual quedo guardada. Eso no es casualidad: es una decision de diseno que se explica en la seccion 5.

3. Paso 1: instalar el SDK

Firebase se agrega con **Swift Package Manager**, el gestor de dependencias que ya trae Xcode. En la interfaz seria *File > Add Package Dependencies*, pegando la URL del repositorio de Firebase.

El SDK de Firebase es un **monorepo**: un solo paquete que contiene muchos productos (Database, Firestore, Analytics, Auth, Storage, Messaging...). Al agregarlo, Xcode pregunta cuales quieres. Este proyecto usa **solo uno**:

Producto	Se incluye	Por que
FirebaseDatabase	Si	Es el que da acceso a Realtime Database

Producto	Se incluye	Por que
FirebaseAnalytics	No	No aporta nada a la app y arrastra GoogleAppMeasurement
Firestore, Auth, Storage...	No	No se usan

Aunque solo se elija un producto, SPM descarga las dependencias que ese producto necesita. Por eso en el proyecto quedan fijados **13 paquetes** (GoogleUtilities, leveldb, nanopb, Promises, etc.). Es normal: son piezas internas del SDK, no hay que tocarlas.

Que queda registrado en el proyecto

La eleccion se guarda en dos archivos:

- `project.pbxproj`: la referencia al paquete y la version pedida (`upToNextMajorVersion 12.18.0`, o sea "cualquier 12.x, nunca 13").
- `Package.resolved`: las versiones **exactas** que se descargaron. Este archivo se commitea a propocito, para que el proyecto compile igual en cualquier otra maquina.

Si clonas el repo en otra Mac. Xcode va a tardar la primera vez: esta descargando los 13 paquetes. No hay que instalar nada a mano ni correr comandos; con abrir el proyecto alcanza.

4. Paso 2: el plist y el arranque

GoogleService-Info.plist

Es un archivo que **genera la consola de Firebase** cuando registras la app. No se escribe a mano. Contiene las credenciales publicas del proyecto y, sobre todo, la URL de la base:

```
BUNDLE_ID    com.inventario.app
PROJECT_ID   inventario-ios
DATABASE_URL  https://inventario-ios-default-rtdb.firebaseio.com
API_KEY      (identificador de cliente, no es un secreto)
```

El BUNDLE_ID tiene que coincidir. Al registrar la app en la consola hay que escribir exactamente el bundle id del proyecto (`com.inventario.app`). Si no coincide, el plist no sirve y Firebase no arranca.

El archivo va en `Resources/` y entra al bundle de la app automaticamente. **Importante:** la URL de la base no esta escrita en ningun lado del codigo Swift. Sale siempre de este archivo, asi que cambiar de proyecto Firebase es reemplazar el plist y nada mas.

FirebaseApp.configure()

El SDK necesita arrancar antes de que alguien lo use. Eso se hace en el `AppDelegate`, que es lo primero que corre al abrir la app:

App/AppDelegate.swift

```
import FirebaseCore
import UIKit

@main
class AppDelegate: UIResponder, UIApplicationDelegate {

    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
            [UIApplication.LaunchOptionsKey: Any]? = nil
    ) -> Bool {
        // Tiene que correr antes que cualquier uso del SDK. Lee el
        // `GoogleService-Info.plist` del bundle por su cuenta: por eso la app no
        // tiene ninguna URL de Firebase escrita en el código.
        FirebaseApp.configure()
        return true
    }
}
```

Son literalmente dos líneas: el `import` y la llamada. `configure()` busca el plist en el bundle, lo lee y deja el SDK listo. Si el plist faltara, la app crashearía en el arranque con un mensaje explícito — no falla en silencio.

Ojo con los ejemplos de la documentación oficial. La documentación de Firebase muestra la versión para **SwiftUI**, que usa `@main struct App` y `@UIApplicationDelegateAdaptor`. Este proyecto es **UIKit**: ya tiene un AppDelegate de verdad, así que la llamada va directo adentro y ese adaptador no hace falta. Copiar el ejemplo de SwiftUI rompería la app.

5. Paso 3: el modelo del dato

Antes de escribir nada hay que decidir que campos tiene un evento. El modelo vive en `EventoBitacora.swift`.

Los dos enums

En vez de guardar textos sueltos, la acción y la entidad son **enums**. Así el compilador impide escribir "creado" en un lado y "creo" en otro:

```
enum AccionBitacora: String {
    case creo = "creo"
    case edito = "edito"
    case elimino = "elimino"

    var descripcion: String {
        switch self {
            case .creo: return "Creo"
            case .edito: return "Edito"
            case .elimino: return "Elimino"
        }
    }
}

enum EntidadBitacora: String {
    case producto = "Producto"
    case categoria = "Categoria"
    case proveedor = "Proveedor"
}
```

El : String le da a cada caso un valor crudo, que es lo que efectivamente viaja a Firebase. descripcion es aparte porque lo que se guarda y lo que se muestra no tienen por que ser iguales.

Los campos del evento

```
struct EventoBitacora {
    let accion: AccionBitacora
    let entidad: EntidadBitacora
    let nombre: String // nombre del registro afectado
    let usuario: String // quien lo hizo
    let fechaISO: String // fecha legible si mirás el arbol en la consola
    let timestamp: Double // segundos desde 1970; es por lo que se ordena

    init(accion: AccionBitacora, entidad: EntidadBitacora,
         nombre: String, usuario: String, fecha: Date = Date()) {
        self.accion = accion
        self.entidad = entidad
        self.nombre = nombre
        self.usuario = usuario
        self.fechaISO = Self.formateador.string(from: fecha)
        self.timestamp = fecha.timeIntervalSince1970
    }
}
```

Dos decisiones que parecen menores y no lo son:

- **Se guarda el nombre, no solo un id.** Si guardáramos productId: 42 y después ese producto se borra, la entrada quedaría diciendo "se elimino el producto 42" sin forma de saber cual era. El nombre congela como se llamaba en el momento del hecho.
- **Hay dos campos de fecha.** timestamp es un número, sirve para ordenar y comparar. fechaISO es texto, y existe para que el dato crudo se entienda si lo mirás desde la consola de Firebase.

Traducción a diccionario

El SDK de Firebase no trabaja con structs de Swift: manda y recibe [String: Any], o sea diccionarios. Así que el modelo sabe convertirse en las dos direcciones.

De Swift a Firebase (para escribir):

```

var comoDiccionario: [String: Any] {
    [
        "accion": accion.rawValue,
        "entidad": entidad.rawValue,
        "nombre": nombre,
        "usuario": usuario,
        "fechaISO": fechaISO,
        "timestamp": timestamp
    ]
}

```

De Firebase a Swift (para leer). Notar que es un `init?`, o sea que puede fallar y devolver `nil`:

```

init?(diccionario: [String: Any]) {
    guard let accionTexto = diccionario["accion"] as? String,
        let accion = AccionBitacora(rawValue: accionTexto),
        let entidadTexto = diccionario["entidad"] as? String,
        let entidad = EntidadBitacora(rawValue: entidadTexto),
        let nombre = diccionario["nombre"] as? String,
        let usuario = diccionario["usuario"] as? String,
        let timestamp = diccionario["timestamp"] as? Double else { return nil }

    self.accion = accion
    self.entidad = entidad
    self.nombre = nombre
    self.usuario = usuario
    self.fechaISO = diccionario["fechaISO"] as? String ?? ""
    self.timestamp = timestamp
}

```

Por que el init puede fallar. Firebase no valida la forma de los datos: cualquiera con acceso a la base podria escribir un nodo con campos raros. Si eso pasa, este `init` devuelve `nil` y esa entrada **se saltea**, en vez de tumbar la lista entera. Un dato corrupto se pierde una fila, no la pantalla.

Fijate que `fechaISO` es el unico campo con `?? ""` en vez de estar en el `guard`: es opcional porque solo sirve para mostrar. Si falta, la entrada sigue siendo utilizable, porque la fecha real se reconstruye del `timestamp`.

6. Paso 4: escribir en Firebase

Toda la conversacion con Firebase esta encerrada en un solo archivo: `BitacoraService.swift`. Ninguna pantalla habla con Firebase directamente.

La referencia a la rama

`Services/BitacoraService.swift`

```

import FirebaseDatabase

final class BitacoraService {

    static let shared = BitacoraService()

    /// Rama del arbol donde vive la bitacora. En Realtime Database no hay
    /// tablas: el arbol entero es un JSON y cada rama se direcciona por su ruta.
    private var referencia: DatabaseReference {
        Database.database().reference(withPath: "bitacora")
    }

    private init() {}
}

```

Realtime Database **no tiene tablas**. Toda la base es un unico JSON gigante, y para trabajar apuntas a una rama por su ruta. Aca la ruta es bitacora, que seria el equivalente conceptual a una tabla.

Database.database() no recibe ninguna URL: la saca de la configuracion que dejo
FirebaseApp.configure() al leer el plist.

El metodo que registra

```
func registrar(_ accion: AccionBitacora, sobre entidad: EntidadBitacora, nombre: String) {
    let evento = EventoBitacora(
        accion: accion,
        entidad: entidad,
        nombre: nombre,
        usuario: SessionManager.shared.username ?? "desconocido"
    )

    // `childByAutoId` genera la clave ordenada por tiempo que el SDK usa para
    // las listas. `setValue` sin completion es exactamente "dispara y
    // olvida": el SDK reintenta solo si el envio no llega.
    referencia.childByAutoId().setValue(evento.comoDiccionario)
}
```

Tres cosas para mirar de cerca:

1. El usuario no se recibe por parametro

Sale de SessionManager.shared.username, adentro del servicio. Si fuera un parametro, cualquier pantalla podria registrar un evento a nombre de otra persona — y en una auditoria eso la invalida por completo.

2. childByAutold()

Genera una clave unica como -P-Rr7FpHTqMvBWxFv. No son aleatorias: llevan el tiempo codificado adentro, asi que ordenar las claves alfabeticamente las ordena cronologicamente. Ademas evita colisiones si dos usuarios escriben al mismo tiempo.

3. setValue sin completion

El metodo **no devuelve error y no tiene callback**. Es a proposito: registrar es un efecto secundario. Si Firebase no contesta, el producto ya se guardo igual y no tendria sentido molestar al usuario con un error sobre algo que a el no le importa. El SDK ademas reintenta por su cuenta si el envio no llega.

La regla que ordena todo esto. La bitacora **nunca** puede romper una operacion del usuario. Por eso registrar no devuelve nada: es imposible que alguien, por error, escriba codigo que cancele un guardado porque fallo la auditoria.

7. Paso 5: leer de Firebase

```
func todos(completion: @escaping (Result<[EventoBitacora], APIError>) -> Void) {
    referencia.queryOrdered(byChild: "timestamp").observeSingleEvent(of: .value) { snapshot in
        var eventos: [EventoBitacora] = []

        for hijo in snapshot.children {
            guard let hijo = hijo as? DataSnapshot,
                  let diccionario = hijo.value as? [String: Any],
                  let evento = EventoBitacora(diccionario: diccionario) else { continue }
            eventos.append(evento)
        }

        // El SDK ya entrega en el hilo principal, pero el orden viene
        // ascendente.
        completion(.success(eventos.reversed()))
    } withCancel: { error in
        completion(.failure(.server(status: 0, message: error.localizedDescription)))
    }
}
```

queryOrdered(byChild:)

Le pide a Firebase que devuelva los hijos ordenados por el campo `timestamp`. Sin esto, el orden no esta garantizado.

observeSingleEvent y no observe

Firebase ofrece dos formas de leer:

Metodo	Comportamiento
<code>observe(...)</code>	Queda escuchando: cada cambio en la base vuelve a llamar al código
<code>observeSingleEvent(...)</code>	Lee una vez y se desuscribe solo

Se usa el segundo porque la pantalla no necesita actualizarse en vivo, y porque una suscripcion viva hay que darla de baja al salir de la pantalla — si te olvidas, queda escuchando para siempre y perdes memoria. Para refrescar alcanza con deslizar hacia abajo.

El recorrido del snapshot

`snapshot` es el nodo `bitacora` entero, y `snapshot.children` son las entradas. De cada una se saca el diccionario y se lo pasa al `init?` del modelo. El `continue` del `guard` es el que implementa el "saltar la entrada corrupta".

reversed()

`queryOrdered` devuelve de menor a mayor, o sea lo mas viejo primero. En pantalla interesa lo ultimo primero, asi que se invierte.

Sobre el rendimiento. Este metodo baja la bitacora completa cada vez. Para una app de curso con decenas o cientos de eventos esta perfecto. Si algun dia fueran miles, la solucion es agregar `.queryLimited(toLast: 50)` para traer solo los ultimos.

8. Paso 6: donde se disparan los eventos

El servicio no se llama solo. Hay **seis puntos** en la app donde el usuario modifica datos, y en cada uno se agrego una linea. Ejemplo con categorias:

Features/Categorias/CategoriaFormViewModel.swift

```
func guardar(nombre: String, descripcion: String) -> String? {
    // ... validaciones ...

    let esAlta = categoria == nil

    if let categoria {
        service.actualizar(categoria, nombre: nombreLimpio, descripcion: descripcionFinal)
    } else {
        categoria = service.crear(nombre: nombreLimpio, descripcion: descripcionFinal)
    }

    BitacoraService.shared.registrar(esAlta ? .creo : .edito,
                                    sobre: .categoria, nombre: nombreLimpio)

    return nil
}
```

El mismo formulario sirve para alta y para edicion, y se distinguen por si categoria ya existia. Por eso esAlta se calcula **antes** del if: despues de crear, la variable ya no es nil y la pregunta daria siempre lo mismo.

Y el registro va **despues** de guardar, no antes: la bitacora deja constancia de lo que efectivamente paso, no de lo que se iba a intentar.

El caso de las bajas: un detalle facil de arruinar

Features/Productos/ProductoListViewModel.swift

```
func eliminar(en indice: Int) {
    guard productos.indices.contains(indice) else { return }
    let producto = productos[indice]

    // El nombre se lee ANTES de marcar la baja: despues la fila puede no
    // estar mas y la entrada de la bitacora quedaria sin con que nombrarla.
    let nombre = producto.nombre ?? "(sin nombre)"

    ProductoService().marcarParaEliminar(producto)
    BitacoraService.shared.registrar(.elimino, sobre: .producto, nombre: nombre)
    cargarLocales()
}
```

El orden importa. Si leyeras producto.nombre despues de marcarParaEliminar, el objeto podria estar ya invalidado y obtendrias vacio o un crash. La entrada quedaria como "Elimino Producto · (sin nombre)", justo el dato que mas interesa cuando alguien borro algo.

Los seis puntos son: alta/edicion y baja de **producto**, de **categoria** y de **proveedor**.

9. Paso 7: la pantalla

La app usa el patron **MVVM**: el ViewModel pide los datos y avisa por closures; el ViewController solo dibuja.

El ViewModel

Features/Bitacora/BitacoraListViewModel.swift

```
final class BitacoraListViewModel {

    var onCambio: (() -> Void)?
    var onError: ((String) -> Void)?
    var onLoadingChanged: ((Bool) -> Void)?

    private(set) var eventos: [EventoBitacora] = []
    private let service = BitacoraService.shared

    var estaVacio: Bool { eventos.isEmpty }

    func cargar() {
        onLoadingChanged?(true)
        service.todos { [weak self] resultado in
            guard let self else { return }
            self.onLoadingChanged?(false)
            switch resultado {
            case .success(let eventos):
                self.eventos = eventos
                self.onCambio?()
            case .failure(let error):
                self.onError?(error.errorDescription ?? "No se pudo cargar la bitacora")
            }
        }
    }
}
```

[weak self] evita un ciclo de retencion: sin eso, el ViewModel se quedaria vivo esperando una respuesta aunque el usuario ya se haya ido de la pantalla.

private(set) significa que cualquiera puede *leer* eventos, pero solo el ViewModel puede *modificarlo*. La pantalla no puede ensuciar el estado por accidente.

La celda

La pantalla **reusa** la celda del catalogo, porque la estructura es la misma: un titulo y un detalle debajo.

Features/Bitacora/BitacoraListViewController.swift

```
celdaCatalogo.configurar(
    nombre: evento.resumen,                // "Edito Producto - Taladro"
    detalle: "\(evento.usuario) - \(evento.fechaLegible)", // "admin - 19/08/2026 20:37"
    chip: nil                             // sin chip
)
```

resumen y fechaLegible son propiedades calculadas del modelo. Que el modelo sepa como presentarse evita repetir ese armado de texto en cada pantalla que lo muestre.

Quien puede verla

La fila vive en **Cuenta > Bitacora**, dentro de la seccion Administracion, y solo aparece si el usuario es **ADMIN**. Una auditoria es justamente para quien controla lo que hacen los demas.

10. Como se ve el dato en Firebase

Asi queda el arbol en la consola (*Compilacion > Realtime Database > Datos*):

```

inventario-ios-default-rtdb
■ ■ ■ bitacora
  ■ ■ ■ -P-Ro3ZR00Whl9W3WywG
  ■ ■ ■ accion: "creo"
  ■ ■ ■ entidad: "Categoria"
  ■ ■ ■ fechaISO: "2026-08-20T01:36:46Z"
  ■ ■ ■ nombre: "Prueba Bitacora"
  ■ ■ ■ timestamp: 1787189806.866634
  ■ ■ ■ usuario: "admin"
  ■ ■ ■ -P-Rr7FpHThqMvBWxwFv
  ■ ■ ■ accion: "elimino"
  ■ ■ ■ ...

```

Cada clave rara es la que genero `childByAutoId()`. Los seis campos son exactamente los que arma como `Diccionario`.

Truco para demostrar que funciona. Deja la consola abierta en esa pantalla y crea una categoria desde la app. El nodo nuevo aparece **solo, sin recargar**, y se resalta unos segundos: la consola escucha en tiempo real. Es la forma mas directa de mostrar que la app esta escribiendo ahi.

11. Como verificarlo desde la terminal

Realtime Database tambien tiene API REST, asi que se puede consultar sin abrir la consola. Es muy comodo para verificar rapido:

Terminal

```

# Ver toda la bitacora
curl -s "https://inventario-ios-default-rtdb.firebaseio.com/bitacora.json"

# Una bitacora vacia responde: null

```

Esto sirve para confirmar que un evento llego **de verdad** a la base, sin depender de lo que muestre la pantalla — si la app y el curl coinciden, el circuito completo funciona.

12. Decisiones de diseno, resumidas

Decision	Motivo
Guardar una entidad nueva (no copiar productos)	Copiar datos del backend crearia dos fuentes de verdad para el mismo dato
Realtime Database (no Firestore)	El dato es una lista plana que solo crece; Firestore agrega complejidad que no se usa
Solo FirebaseDatabase (sin Analytics)	Analytics no aporta a la app y arrastra dependencias pesadas
registrar() sin completion	La auditoria nunca puede romper una operacion del usuario
El usuario sale de SessionManager	Si fuera parametro, una pantalla podria registrar a nombre de otro
Se guarda el nombre, no solo el id	La entrada tiene que seguir siendo legible cuando el registro ya no exista
Leer el nombre antes de borrar	Despues de marcar la baja el objeto puede estar invalidado

Decision	Motivo
observeSingleEvent	La pantalla no necesita tiempo real, y una suscripcion viva hay que darla de baja
init? que devuelve nil	Una entrada corrupta se saltea en vez de tumbar la lista entera
Todo encerrado en BitacoraService	Ninguna pantalla habla con Firebase; cambiar de backend toca un solo archivo

Sobre el ultimo punto, un caso real. Esta implementacion se escribio primero con la **API REST** de Firebase, sin SDK. Cuando hubo que migrar al SDK oficial, el cambio fue reescribir BitacoraService por dentro: ni el modelo, ni los seis puntos que registran, ni la pantalla se enteraron. Eso es lo que compra encapsular bien.

13. Seguridad y limitaciones

Las reglas de la base estan abiertas

La base se creo en **modo de prueba**: cualquiera que conozca la URL puede leer y escribir. Firebase las cierra solo a los 30 dias de creada, y cuando eso pase la app va a empezar a fallar al escribir.

Para un trabajo de curso alcanza, pero conviene tener presente:

- **No guardar datos reales de clientes** en esa base.
- **Borrar el proyecto de Firebase** al cerrar la materia.

El plist esta en el repositorio publico

Eso es lo normal en iOS: el `GoogleService-Info.plist` esta pensado para viajar dentro de la app, y su `API_KEY` es un identificador de cliente, no una contrasena. Cualquiera que descargue una app de iPhone puede sacarlo.

La idea importante. Lo que protege una base de Firebase son **las reglas**, no esconder el archivo de configuracion. Con reglas abiertas, el riesgo existe aunque el plist fuera secreto; con reglas bien puestas, publicarlo no es problema.

La bitacora no funciona sin conexion

A diferencia del resto de la app, que es *offline-first*, los eventos van a la red en el momento. Es deliberado: un registro de auditoria que se encola en el telefono y se sube cuando quiere no sirve como rastro confiable. Si no hay conexion, el SDK reintenta solo; si aun asi no llega, se pierde ese evento — pero la operacion del usuario nunca se ve afectada.